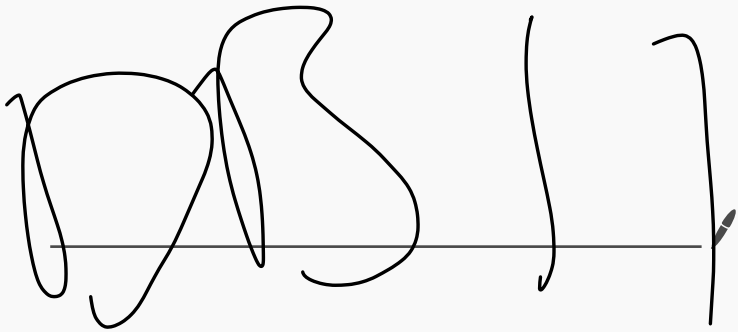


lecture 6+7

1- Heuristic based rules (may work or not)

1- Perform projection early (reduce attributes)
2- Perform selection early (reduce tuples)
3- make the most restrictions first

2- cost based optimization

$T(R)$ = tuples
$S(R)$ = # of bytes in each R tuple
$B(R)$ = # of blocks (pages)
$V(R,A)$ = distinct values in R

$W = \sigma_{Z=val}(R)$

$$\frac{T(R)}{V(R,Z)} \text{ || Dom}$$

$\sigma_{Z \geq val}$

$\frac{1}{2}T$	$\frac{1}{3}T$	$\frac{\text{max-val+1}}{\text{max-min+1}}$

and = ($\wedge$)

or = ($+$)

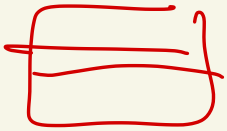
$\boxtimes$

$$\frac{T(R2)\,T(R1)}{\max(V(R1,A),V(R2,A))}$$

lecture 8 physical plan

lecture 9 cost of physical plan

1- contigous mean there is pages →



paramatares

B(R) = blocks having R tuples
f(R) = max tuples per block
M = number of blocks available

Types of joins

1- iteration join

nested loops → most expensive

R1 (read R1 + R2)

2- merge join

Sorted: Read R1 cost + R cost
not sorted (have to sort first)

4R1 + 4R2

3- Join with index

3 cases

R1 = 10000
R2 = 5000

1- R1.C primary key R2.C foreign key

R1 fits in memory

So 1 tuple matches

2- both foreign key

3- ~~[illegible]~~ no stats

lecture 10 logs and recovery

1- failure models

Event

desired

coded

undesired

expected

system crash
memory damage
CPU halts

unexpected

disk
stolen

logging techniques

1- undo (do it all again)

2- redo (redo it all)

input
output

buffer for memory

read
write

transaction buffer

lec 11 transactions

Scheduler to schedul process

Same as OS

Serializability → if a schedule can make same output as the serial one

conflict serizible → serial

non conflicting pairs	conflicting pairs
$r_i(x), r_j(y)$	$r_i(x), w_i(x)$
$r_i(x), w_j(y)$	$w_i(x), w(x)$
$w_i(x), w_j(y)$	$r_i(x), w_j(x)$
	$w_i(x)$ $r_j(x)$

if changed called conflict equivalent

Locks

two phase locks (lock any other resource before unlocking)

shared lock (when they read multiple stuff)

exclusive lock (when writing)

upgrading locks (start with shared then upgrade to exclusive)

🧠 **Algorithm:**

1. Begin at the **root**.
2. If you're **at the target element**, request a **shared (S)** or **exclusive (X)** lock.
3. If you're **not at the target**, request an **intention lock**:
   - `IS` (Intention Shared) if you'll request S later
   - `IX` (Intention Exclusive) if you'll request X later
4. If the lock is **granted**, move down. If **not**, **wait**.

Algorithm for tree locking